VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES AND ALGORITHMS - CO2003

ASSIGNMENT 2

IMPLEMENTING SEARCH MECHANISMS IN VECTORSTORE USING TREE DATA STRUCTURES

HO CHI MINH CITY, 11/2025

ASSIGNMENT'S SPECIFICATION

Version 1.0

1 Assignment's outcome

After completing this assignment, students will be able to:

- Master object-oriented programming (OOP).
- Develop tree-based data structures.
- Use tree-based data structures to implement a VectorStore that supports multiple search mechanisms.

2 Introduction

In Assignment 2, students are required to implement a VectorStore that uses tree-based data structures to efficiently store and query multidimensional vectors. Specifically, an AVL tree is employed as the primary index, organizing records by their Euclidean distance to a reference vector in order to maintain balance and support insert/search/delete operations with logarithmic complexity, while also allowing in-order traversal by distance to serve range queries.

In addition, the system includes a secondary index implemented with a red–black tree (RBT), which sorts vectors by their norm. This serves as a pre-filtering mechanism to quickly select candidate vectors with similar magnitudes, thereby reducing the number of true distance computations when finding the top-k nearest neighbors. The design of these two coordinated indices reflects a simple yet efficient VectorStore model, suitable for semantic search, content recommendation, and knowledge management based on embeddings.

The objectives of this assignment are to help students:

- Strengthen object-oriented programming skills through the design of well-structured tree and vector record classes with clear responsibilities.
- Understand and implement the detailed mechanisms of two balanced tree structures (AVL and RBT), including color/height rules, rotations, and the rebalancing process during insert/delete operations.
- Apply tree data structures in a practical VectorStore: use an AVL tree as the primary index based on the distance to a reference vector, and use an RBT as a norm-based index for pre-filtering before ranking.

3 Description

3.1 AVL Tree

AVL tree is a self-balancing binary search tree, in which the height difference between the left and right subtrees of every node (called the balance factor) does not exceed 1. The AVL tree guarantees that search, insertion, and deletion operations all have a time complexity of $O(\log n)$ in all cases.

3.1.1 Class AVLNode

The `AVLNode<K, T>` class describes a node in an AVL tree. Each node stores:

3.1.1.1 Attributes:

- `K key`: the key used for comparison (template type `K`).
- `T data`: the data value stored in the node (template type `T`).
- `AVLNode* pLeft`: pointer to the left child node.
- `AVLNode* pRight`: pointer to the right child node.
- `BalanceValue balance`: the balance factor of the node, represented by an enum:

Name	Value	Meaning
`LH`	-1	Left Higher (the left subtree is higher)
`EH`	0	Equal Height (both subtrees are equal in height)
`RH`	1	Right Higher (the right subtree is higher)

3.1.1.2 Methods:

- `AVLNode(const K& key, const T& value)`: Constructor that initializes the node with `key` and `value`; both `pLeft` and `pRight` pointers are assigned to `nullptr`, and `balance` is set to `EH`.
  - **Time complexity**: $O(1)$.

3.1.2 Class AVLTree

3.1.2.1 Attributes:

- `AVLNode<K, T>* root`: Pointer to the root node of the AVL tree. Initialized as `nullptr` when creating an empty tree.

3.1.2.2 Protected methods (internal support):

- `AVLNode<K, T>* rotateRight(AVLNode<K, T>*& node)`
  Performs a right rotation at the given node to rebalance the tree when the left subtree is higher than the right subtree by more than 1.
  - **Input**: The node to be right-rotated.
  - **Output**: The new root node after the right rotation.
  - **Time complexity**: $O(1)$.
- `AVLNode<K, T>* rotateLeft(AVLNode<K, T>*& node)`
  Performs a left rotation at the given node to rebalance the tree when the right subtree is higher than the left subtree by more than 1.
  - **Input**: The node to be left-rotated.
  - **Output**: The new root node after the left rotation.
  - **Time complexity**: $O(1)$.

3.1.2.3 Public methods:

- `AVLTree()`
  Constructor that initializes an empty AVL tree with `root = nullptr`.
  - **Time complexity**: $O(1)$.
- `~AVLTree()`
  Destructor that releases all allocated memory for the tree (all nodes).
  - **Time complexity**: $O(n)$.
- `void insert(const K& key, const T& value)`
  Inserts a node into the AVL tree. The method automatically maintains the balance of the tree.
  - **Input**: The key `key` and the value to be inserted `value`.
  - **Time complexity**: $O(\log n)$.
  - **Note**: If the value already exists, the insertion is not performed.

- `void remove(const K& key)`
  Removes a value from the AVL tree. The method automatically rebalances the tree if necessary. It uses the node with the smallest key in the right subtree as a replacement.
  - **Input**: The key `key` to be removed from the tree.
  - **Time complexity**: $O(\log n)$.
  - **Note**: Removes the first node with the given `key` if it exists. If the value does not exist, no action is taken.
- `bool contains(const T& key) const`
  Checks whether a key exists in the tree.
  - **Input**: The key `key` to be checked.
  - **Output**: `true` if the value exists, `false` otherwise.
  - **Time complexity**: $O(\log n)$.
- `int getHeight() const`
  Returns the height of the AVL tree.
  - **Output**: The height of the tree. Returns 0 if the tree is empty.
  - **Time complexity**: $O(n)$.
- `int getSize() const`
  Returns the number of nodes currently in the tree.
  - **Output**: The number of nodes in the tree.
  - **Time complexity**: $O(n)$.
- `bool empty() const`
  Checks whether the tree is empty.
  - **Output**: `true` if the tree is empty, `false` if it contains at least one node.
  - **Time complexity**: $O(1)$.
- `void clear()`
  Deletes all nodes in the tree and sets `root = nullptr`. Frees all allocated memory.
  - **Time complexity**: $O(n)$.
- `void printTreeStructure() const`
  Prints the structure of the tree level by level (level-order traversal) to visualize the AVL tree (provided).
  - **Time complexity**: $O(n)$.

- `void inorderTraversal(void (*action)(const T&)) const`
  Traverses the tree in in-order (left-root-right) and executes the function `action` on each value.
  - **Input**: Pointer to the function `action` used to process each element.
  - **Time complexity**: $O(n)$.

3.2 Red-Black Tree

3.2.1 Description

A Red-Black Tree is a self-balancing binary search tree in which each node carries a color attribute (either red or black) to control the imbalance of the tree. Therefore, the height of the tree is proportional to $\log n$, and lookup operations achieve a time complexity of $O(\log n)$.

The Red-Black Tree maintains the binary search tree invariant: for every node x, all keys in the left subtree of x are smaller than the key of x, and all keys in the right subtree of x are greater than the key of x.

Color Rules: Let `NIL` denote null leaves (considered black) and `root` denote the root of the tree. A valid Red-Black Tree must simultaneously satisfy the following properties:

- Each node is colored either red or black.
- The root node is always black.
- All `NIL` leaves are considered black.
- If a node is red, then both of its children are black.
- For every node, all simple paths from that node to its descendant `NIL` leaves contain the same number of black nodes. This number is called the **black-height**.

From these properties, it follows that the height of a Red-Black Tree is bounded by a constant multiple of $\log n$; therefore, the *search* and update operations (after rebalancing) both have a worst-case time complexity of $O(\log n)$.

Students may refer to additional resources on Red-Black Trees here: 1, 2, 3

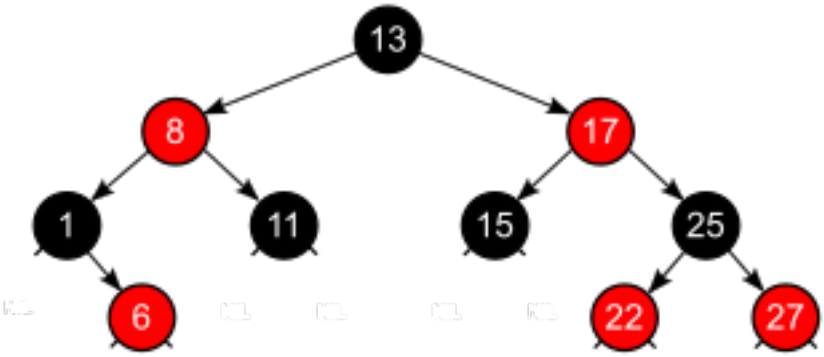

Figure 1: Red-Black Tree

3.2.2 Rules

3.2.2.1 Insertion and Rebalancing When inserting a new key into a Red-Black Tree, first insert it as in a standard binary search tree and color the new node red; then perform rotations and recoloring to maintain the color rules. Let N be the newly inserted node, $P = \text{parent}(N)$ the parent, $G = \text{parent}(P)$ the grandparent, and U the "uncle" of N (the sibling of P). The fixing process follows these cases:

1. **Case 1 (Root):** If N is the root, color N black and finish. This ensures the "root is black" property.
2. **Case 2 (Parent red, Uncle red – recolor):** If P is red and U exists and is also red, recolor P and U to black, and color G red. Then set $N \leftarrow G$ and repeat the fixing process upward to ensure global color balance.
3. **Case 3 (Triangle – rotate first, then relabel):**
   - If P is red, U is black (or does not exist), and the configuration is "left–right" (N is the right child of P, and P is the left child of G), perform a *left rotation* at P; then set $N \leftarrow P$ to transform into a "straight line" configuration and continue fixing.
   - If the configuration is "right–left" (N is the left child of P, and P is the right child of G), perform a *right rotation* at P; then set $N \leftarrow P$ to transform into a "straight line" configuration and continue fixing.
4. **Case 4 (Straight line – rotate at grandparent, then recolor):**
   - If after the previous step (or initially) the configuration is "left–left" (N is the left child of P, and P is the left child of G), perform a *right rotation* at G.
   - If the configuration is "right–right" (N is the right child of P, and P is the right child of G), perform a *left rotation* at G.

After rotation at G: color P (the new parent after rotation) **black** and color G **red**. All color rules are now restored and the process terminates.

Students may consult additional illustrative examples here: 1, 2

3.2.2.2 Deletion and Rebalancing Deleting a node Z involves two steps: (i) If Z has two children, swap its data with its *predecessor* (the largest node in its left subtree), then set Z to that predecessor node; (ii) Delete Z when it has at most one non-`NIL` child.

Let X be the child node (possibly `NIL`) that is "moved up" to replace Z. If the removed node was red, the process terminates; if it was black, a "double black" violation occurs at X and must be fixed using the following cases (denote $P = \text{parent}(X)$, S as the sibling of X, and S_L, S_R as the left and right children of S):

1. **Root:** If X becomes the root, color X black and finish.
2. **Red sibling (rotate parent, recolor):** If S is red, rotate around P so that S becomes the new parent of P, recolor S black and P red. Now X has a black sibling; continue to cases 3–5.
3. **Black sibling, both children of S black (recolor, push double black upward):** Color S red, set $X \leftarrow P$, and repeat at the higher level. If P is red, recolor P black and terminate.
4. **Black sibling, "near child" red, "far child" black (rotate at S):**
   - If X is a left child of P and S_L is red, S_R black: perform a right rotation at S, recolor S red and S_L black; move to Case 5.
   - Symmetrically, if X is a right child of P and S_R is red, S_L black: perform a left rotation at S, recolor S red and S_R black; move to Case 5.
5. **Black sibling, "far child" red (rotate at P, recolor and finish):**
   - If X is the left child of P and S_R is red: perform a left rotation at P; color S with the old color of P, color P black, and color S_R black; finish.
   - Symmetrically, if X is the right child of P and S_L is red: perform a right rotation at P; color S with the old color of P, color P black, and color S_L black; finish.

3.2.2.3 Searching for a Node in a Red-Black Tree Searching for a node in a Red-Black Tree is identical to searching in a standard binary search tree.

3.2.3 Class RBTNode

3.2.3.1 Attributes An `RBTNode` represents a node of a red-black tree with the following basic fields:

- `K key`: the key used for comparison (template type K)
- `T data`: the data value stored in the node (template type T)
- `Color color`: the color of the node, represented by an enum type.
- `RBTNode* parent`: pointer to the parent node.
- `RBTNode* left`, `RBTNode* right`: pointers to the left/right child nodes.

3.2.3.2 Methods

- `RBTNode(const K& key, const T& value)`: Constructor that creates a new node with the given `key` and `value`, and assigns `color = RED`, `left = right = parent = nullptr`.
  - Time complexity: $O(1)$
- `void recolorToRed()`: Changes the color of the current node to `RED`.
  - Time complexity: $O(1)$
- `void recolorToBlack()`: Changes the color of the current node to `BLACK`.
  - Time complexity: $O(1)$

3.2.4 Class RedBlackTree

3.2.4.1 Attributes

- `RBTNode<K,T>* root`: Pointer to the root node of the red–black tree. Initialized to `nullptr` when the tree is empty.

3.2.4.2 Protected Methods

- `void rotateLeft(RBTNode<K,T>* node)`
  Performs a left rotation at the given `node` to balance the tree.
  - Input: `node` — the node to perform the left rotation on.
  - Output: None.
  - Time complexity: $O(1)$.

  - Note: Preserves the BST invariant; only modifies the local structure.

- `void rotateRight(RBTNode<K,T>* node)`
  Performs a right rotation at the given `node` to locally restructure the tree.
  - Input: `node` — the node to perform the right rotation on.
  - Output: None.
  - Time complexity: $O(1)$.
  - Note: Symmetric to `rotateLeft`.

- `RBTNode<K,T>* lowerBoundNode(const K& key) const`
  Finds the first node whose key is $\geq$ `key`.
  - Input: `key`.
  - Output: Pointer to the matching node or `nullptr` if not found.
  - Time complexity: $O(\log n)$.

- `RBTNode<K,T>* upperBoundNode(const K& key) const`
  Finds the first node whose key is $>$ `key`.
  - Input: `key`.
  - Output: Pointer to the matching node or `nullptr`/`NIL`.
  - Time complexity: $O(\log n)$.

3.2.4.3 Public Methods

- `RedBlackTree()`
  Constructor that initializes an empty tree.
  - Time complexity: $O(1)$.

- $\sim$ `RedBlackTree()`
  Destructor that releases all nodes allocated by the user.
  - Input: None.
  - Time complexity: $O(n)$.

- `bool empty() const`
  Checks whether the tree is empty.
  - Output: `true` if empty, `false` otherwise.
  - Time complexity: $O(1)$.

- `int size() const`
  Returns the current number of nodes.

  - Input: None.
  - Output: The total number of nodes in the tree.
  - Time complexity: $O(n)$.

- `void clear()`
  Deletes all nodes and resets the `root` to empty.
  - Time complexity: $O(n)$.

- `void insert(const K& key, const T& value)`
  Inserts a `(key, value)` pair into the tree. If the `key` already exists, no action is taken.
  - Input: `key`, `value`.
  - Time complexity: $O(\log n)$.

- `void remove(const K& key)`
  Removes the first node with the given `key`, if it exists. The method automatically rebalances the tree using the replacement of the node with the largest key in the left subtree (if necessary).
  - Input: `key`.
  - Time complexity: $O(\log n)$.
  - Note: If the key does not exist, no operation is performed.

- `RBTNode<K, T>* find(const K& key) const`
  Searches and returns a pointer to the first node with the given `key`.
  - Input: `key`.
  - Output: `RBTNode<K, T>*` pointer to the data, or `nullptr` if not found.
  - Time complexity: $O(\log n)$.

- `bool contains(const K& key) const`
  Checks whether a given `key` exists in the tree.
  - Input: `key`.
  - Output: `true`/`false`.
  - Time complexity: $O(\log n)$.

- `RBTNode<K, T>* lowerBound(const K& key, bool& found) const`
  Returns the pointer to the node with the **smallest** key $\geq$ `key`.
  - Input: `key`.
  - Output: `found = true` and the lower bound key if it exists; `found = false` if not. Pointer to the node with the smallest key satisfying the condition or `nullptr` if none exists.

  - Time complexity: $O(\log n)$.

- `RBTNode<K, T>* upperBound(const K& key, bool& found) const`
  Returns the pointer to the node with the **smallest** key $>$ `key`.
  - Input: `key`.
  - Output: `found = true` and the upper bound key if it exists; `found = false` if not. Pointer to the node with the smallest key satisfying the condition or `nullptr` if none exists.
  - Time complexity: $O(\log n)$.

- `void printTreeStructure() const`
  Prints the tree structure level by level (level-order traversal) to visualize the red-black tree (provided as utility).
  - **Time complexity**: $O(n)$.

3.3 Vector Store and Search Mechanism in VectorStore

3.3.1 Introduction

VectorStore is a specialized data structure for storing and querying multi-dimensional vectors. The AVL tree is used as the primary index, ordering records by their distance to a reference vector to ensure stable insert/search/delete operations and enable traversal in distance order. Additionally, the system integrates a red–black tree (RBT) as a "norm index," sorting by the Euclidean norm of each vector. During queries, the RBT provides fast candidate filtering through lower-bound/upper-bound operations on the norm axis, keeping only those vectors whose norms lie within a window around the query's norm before performing precise scoring and selecting the top-k. This significantly reduces computation and accelerates nearest-neighbor search in embedding-based AI/ML applications.

3.3.2 Architecture and Operating Principle

3.3.2.1 Concept of a Distance-Centered VectorStore VectorStore does not organize vectors by key ID or a particular dimension, but by their Euclidean distance to a designated reference vector. This approach offers several advantages:

- Vectors that are close in distance are stored near each other in the tree, optimizing nearest-neighbor queries.

- Enables efficient range queries by traversing the tree in distance order.
- Supports approximate nearest-neighbor search by querying only subtrees within a suitable distance range.
- Incorporates a secondary index using a red–black tree (RBT) sorted by the vector's "Euclidean norm," allowing fast candidate filtering via `lower_bound/upper_bound` on the [*norm*] axis to obtain a subset around the query's [*norm*] before precise scoring, thus greatly reducing computations and improving top-k query speed.

3.3.2.2 Two Key Vector Concepts

- **Reference Vector**: The base vector provided by the user when initializing the VectorStore. It serves as the reference point for computing distances from all other vectors. The reference vector may not exist within the store itself.
- **Root Vector of the AVL Tree**: This is **not** the reference vector, but rather a **vector within the store** selected such that its distance to the reference vector is closest to the **average distance** from the reference vector to all vectors in the store. This root vector is used as the root node of the AVL tree.

3.3.3 Class VectorRecord

Each vector is encapsulated into a record to manage both its data and related metadata:

- `int id`: A unique identifier for the vector (not used as a sorting key in the AVL tree). The `id` increases according to the rule: the `id` of the newest record equals the current maximum `id` in the store plus one.
- `string rawText`: The original text string before being mapped into a vector.
- `int rawLength`: The length of the original text string.
- `vector<float>* vector`: A pointer to the numerical vector data.
- `double distanceFromReference`: The distance from the reference vector to this vector (used for computing the average distance).
- `double norm`: The "Euclidean norm" of the numerical vector. Used to build the red–black tree based on the norm.

3.3.4 Class VectorStore

3.3.4.1 Class Attributes

- `AVLTree<double, VectorRecord>* vectorStore`: An AVL tree that stores `VectorRecord` entries. The `distanceFromReference` value serves as the key.
- `RedBlackTree<double, VectorRecord>* normIndex`: A red–black tree that stores `VectorRecord` entries, serving as a secondary index to support search. The `norm` value serves as the key.
- `vector<float>* referenceVector`: The base vector provided by the user during initialization, used as the reference point for computing the distance of all other vectors. The reference vector does not necessarily reside within the store.
- `VectorRecord* rootVector`: The record of the vector selected as the root vector of the AVL tree (the vector within the store whose distance is closest to the average distance from the reference vector).
- `int dimension`: The fixed number of dimensions for all vectors in the store.
- `int count`: The number of vector records currently stored.
- `double averageDistance`: The average distance from the reference vector to all vectors currently in the store. Used as the criterion for selecting the root vector.
- `vector<float>* (*embeddingFunction)(const string&)`: A function pointer that maps a text string into a multi-dimensional numerical vector.

3.3.4.2 Public Methods

Initialization and Basic Management

- `VectorStore(int dimension = 512, vector<float>* (*embeddingFunction)(const string&), const vector<float>& referenceVector)`
  Initializes an empty VectorStore with a fixed dimension, embedding function, and reference vector. The store remains empty until the first vector is added.
  - **Exception**: None.
  - **Complexity**: $O(1)$.
- `~VectorStore()`
  Destructor that releases all memory allocated for the AVL Tree, vector records, and related data.
  - **Complexity**: $O(n)$.
- `int size()`
  Returns the number of vectors currently stored.
  - **Complexity**: $O(1)$.

- `bool empty()`
  Checks whether the store is empty.
  - **Complexity**: $O(1)$.
- `void clear()`
  Deletes all vectors and associated metadata from both the AVL and red–black trees, but retains the reference vector.
  - **Complexity**: $O(n)$.

Preprocessing and Data Management

- `vector<float>* preprocessing(string rawText)`
  Preprocesses the input text into a numerical vector. Calls the embedding function to map the text, then normalizes the vector's dimension (by truncating or padding).
  - **Requirements**:
    1. Call `embeddingFunction` to map the `rawText` string into a vector.
    2. If the number of dimensions exceeds `dimension`, truncate the trailing elements.
    3. If the number of dimensions is smaller than `dimension`, append zero-valued elements (post-padding).
  - **Complexity**: $O(d)$, where d is the number of dimensions.
- `void addText(string rawText)`
  Adds a new record to the store from the original text string. The method performs:
  1. Uses preprocessing to convert text into a vector.
  2. Computes the distance from the reference vector to the newly created vector.
  3. Updates the average distance.
  4. Computes the "Euclidean norm" of the vector.
  5. If the store is empty, sets this vector as the root vector.
  6. If the store is not empty:
     - Inserts the vector into the AVL tree. If the new vector's distance is closer to the average distance than the current root's, restructures the tree with a new root vector.
     - Inserts the vector into the red–black tree, ensuring balance after insertion.
  - **Complexity**: $O(\log n)$ in the average case, $O(n \log n)$ if root restructuring is required.

- `VectorRecord* getVector(int index)`
  Retrieves a reference to the vector at position `index` when traversing the **AVL tree** in in-order sequence (starting from 0).
  - **Exception**: Throws `out_of_range("Index is invalid!")` if the index is invalid.
  - **Complexity**: $O(n)$.
- `string getRawText(int index)`
  Retrieves the original text string at position `index` during an in-order traversal of the **AVL tree**.
  - **Exception**: Throws `out_of_range("Index is invalid!")` if the index is invalid.
  - **Complexity**: $O(n)$.
- `int getId(int index)`
  Retrieves the identifier (`id`) of the vector at position `index` during an in-order traversal of the **AVL tree**.
  - **Exception**: Throws `out_of_range("Index is invalid!")` if the index is invalid.
  - **Complexity**: $O(n)$.
- `bool removeAt(int index)`
  Removes the vector and its metadata at position `index` during an in-order traversal of the **AVL tree**, frees its memory, and deletes it from the AVL tree. If the removed vector is the root vector, finds the new vector closest to the updated average distance to serve as the new root. The same vector must also be deleted from the red–black tree to maintain consistency.
  - **Exception**: Throws `out_of_range("Index is invalid!")` if the index is invalid.
  - **Complexity**: $O(n + \log n) = O(n)$.

Reference vector and embedding function management

- `void setReferenceVector(const vector<float>& newReference)`
  Change the reference vector to a new vector. This method will recompute the distances from the new reference vector to all existing vectors, update the average distance, find the new vector closest to the average distance to serve as the new root vector, and reconstruct the AVL tree.
  - **Complexity**: $O(n \log n)$.
- `VectorRecord* getReferenceVector() const`
  Return a pointer to the current reference vector (provided by the user).

  - **Complexity**: $O(1)$.

- `VectorRecord* getRootVector() const`
  Return a pointer to the current root vector of the AVL tree.
  - **Complexity**: $O(1)$.
- `double getAverageDistance() const`
  Return the average distance from the reference vector to all vectors in the store.
  - **Complexity**: $O(1)$.
- `void setEmbeddingFunction(vector<float>* (*newEmbeddingFunction)(const string&))`
  Update the mapping function that converts text into vectors.
  - **Complexity**: $O(1)$.

Traversal and iteration

- `void forEach(void (*action)(vector<float>&, int, string&))`
  Traverse all records in the AVL Tree in in-order sequence (increasing distance from the root vector) and execute the `action` function on each record.
  - **Complexity**: $O(n)$.
- `std::vector<int> getAllIdsSortedByDistance() const`
  Return a list of all vector record ids sorted by increasing distance from the root vector of the AVL tree.
  - **Output**: A vector containing the ids.
  - **Complexity**: $O(n)$.
- `std::vector<VectorRecord*> getAllVectorsSortedByDistance() const`
  Return a list of all vector records sorted by increasing distance from the root vector of the AVL tree.
  - **Output**: A vector containing record pointers.
  - **Complexity**: $O(n)$.

Distance metrics

- `double cosineSimilarity(const vector<float>& v1, const vector<float>& v2)`
  Compute the cosine similarity between two vectors using the formula:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

  - **Complexity**: $O(d)$.

- `double l1Distance(const vector<float>& v1, const vector<float>& v2)`
  Compute the Manhattan distance between two vectors:

$$d(\vec{A}, \vec{B}) = \sum_{i=1}^{d} |A_i - B_i|$$

  - **Complexity**: $O(d)$.

- `double l2Distance(const vector<float>& v1, const vector<float>& v2)`
  Compute the Euclidean distance between two vectors:

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^{d} (A_i - B_i)^2}$$

  - **Complexity**: $O(d)$.

Estimating threshold `D` from `k`

- `double estimateD_Linear(const vector<float>& query, int k, double averageDistance, const vector<float> reference, double c0_bias = 1e-9, double c1_slope = 0.05)`
  Estimate the radius D using a linear-in-k model based on existing statistics from the VectorStore.

  - **Input:**
    - `query`: query vector.
    - `k`: number of nearest neighbors to find.
    - `averageDistance`: $a = \text{mean}\{\|v - r\|\}$ currently stored in the repository.
    - `reference`: reference vector r.
    - `c0_bias`, `c1_slope`: simple adjustment coefficients (default 1e−9 and 0.05).
  - **Output:** Value $D > 0$
  - **Formula:** $d_r = \|\,\texttt{query} - \texttt{reference}\,\|\,, \quad D = \big|\,d_r - \texttt{averageDistance}\,\big| + \texttt{c1\_slope} \cdot \texttt{averageDistance} \cdot k + \texttt{c0\_bias}\,.$

Nearest Neighbor Search

- `int findNearest(const vector<float>& query, string metric = "cosine")`

Find the `VectorRecord` closest to the query vector according to the given metric. Supported metrics: `cosine`, `euclidean`, `manhattan`.

- **Output**: Returns the id of the nearest `VectorRecord`.
- **Exception**: Throws `invalid_metric()` if the metric is invalid.
- **Complexity**: $O(n \times d)$.

`int* topKNearest(const vector<float>& query, int k, string metric = "cosine")`
Find the k nearest `VectorRecord` objects to the query vector and return a dynamic array of `id`s. The method utilizes a red-black tree index by norm to prefilter candidates before computing the actual distances and ranking them.

- **Output:** A dynamic `int` array containing `k` `id`s in ascending order of distance, with exact size `k` if enough candidates exist; if fewer than `k` candidates are found, returns the actual number obtained.
- **Exception:** Throws `invalid_metric()` if the `metric` is invalid; throws `invalid_k_value()` if `k` ≤ 0 or `k` exceeds the store size.
- **Complexity:** $O(\log n + m \cdot d + m \log k)$, where n is the number of vectors, d is the dimensionality, and m is the number of candidates after red-black tree filtering (typically $m \ll n$).
- **Execution steps**:
  - **Step 1**: Compute the Euclidean norm of the query. Calculate $n_q = \|q\|$.
  - **Step 2**: Estimate the radius D from k using the `estimateD_Linear` method.
  - **Step 3**: Filter using the red-black tree (`normIndex`). Use `lower_bound`($n_q - D$) and `upper_bound`($n_q + D$) on the red-black tree to extract the candidate range by norm, creating a candidate set of size m.
    Print the value of `m` in the format: `"Value m: <m>"`.
  - **Step 4**: Compute actual distances on the candidates. For each candidate in the range, compute the true distance according to the chosen `metric`.
  - **Step 5**: Select the top-k and return a dynamic `int` array of size `k` sorted by ascending distance.

Range Query

- `int* rangeQueryFromRoot(double minDist, double maxDist) const`
  Find all vectors whose distance from the AVL tree's root vector lies within the range $[minDist, maxDist]$.
  - **Output**: List of ids of the matching vectors.

  - **Complexity**: $O(k)$ where k is the number of vectors within the range.

- `int* rangeQuery(const vector<float>& query, double radius, string metric = "cosine") const`
  Find all vectors in the AVL tree whose distance/similarity from the query vector lies within the given radius under the specified metric.
  - **Output**: List of ids of the matching vectors.
  - **Exception**: Throws `invalid_metric()` if the metric is invalid.
  - **Complexity**: $O(n \times d)$.

- `int* boundingBoxQuery(const vector<float>& minBound, const vector<float>& maxBound) const`
  Find all vectors in the AVL tree that lie within the bounding box defined by two points `minBound` and `maxBound`.
  - **Output**: List of ids of matching vectors.
  - **Complexity**: $O(n)$.

Advanced Utility Methods

- `double getMaxDistance() const`
  Find the maximum distance from the root vector to any vector in the store.
  - **Complexity**: $O(n)$.

- `double getMinDistance() const` Find the minimum distance from the root vector.
  - **Complexity**: $O(\log n)$.

- `VectorRecord computeCentroid(const std::vector<VectorRecord*>& records) const`
  Compute the centroid of a set of vectors by averaging each dimension.
  - **Complexity**: $O(m \times d)$ where m is the number of vectors and d is the dimensionality.

- `VectorRecord* findVectorNearestToDistance(double targetDistance) const`
  Find the vector in the store whose distance from the reference vector is closest to the target value `targetDistance`.
  - **Complexity**: $O(n)$.

3.4 Root Vector Determination Mechanism

3.4.1 Root Vector Search Algorithm

1. Compute the distance from the reference vector to each vector in the store.
2. Compute the average distance:

$$\text{avgDist} = \frac{1}{n} \sum_{i=1}^{n} \text{distance}_i$$

3. Find the vector whose distance is closest to avgDist:

$$\text{rootVector} = \arg\min_i |\text{distance}_i - \text{avgDist}|$$

4. Set the found vector as the root vector of the AVL tree.

3.4.2 Illustrative Example

Assume a VectorStore with reference vector $\text{ref} = (0, 0, 0)$ containing 5 vectors:

$$v_1 = (1.0, 0, 0), \quad d_1 = 1.0 \tag{1}$$
$$v_2 = (2.0, 0, 0), \quad d_2 = 2.0 \tag{2}$$
$$v_3 = (3.0, 0, 0), \quad d_3 = 3.0 \tag{3}$$
$$v_4 = (4.0, 0, 0), \quad d_4 = 4.0 \tag{4}$$
$$v_5 = (5.0, 0, 0), \quad d_5 = 5.0 \tag{5}$$

Average distance:

$$\text{avgDist} = \frac{1 + 2 + 3 + 4 + 5}{5} = 3.0$$

The vector whose distance is closest to 3.0 is v_3 (since $|3.0 - 3.0| = 0$). Therefore, v_3 is chosen as the root vector of the AVL tree.

The AVL tree will be constructed with v_3 as the root, and other vectors arranged according to their distances from v_3:

- v_1, v_2: Left subtree (shorter distance from v_3).
- v_4, v_5: Right subtree (greater distance from v_3).

3.5 Conclusion

The VectorStore with an AVL Tree provides an efficient and optimized method for storing, searching, and querying multidimensional vector data. By intelligently selecting the root vector (the one closest to the average distance), the system automatically optimizes the tree structure to best serve queries in vector space. The AVL Tree guarantees logarithmic complexity for basic operations, while traversal based on the distance from the root enables query range optimization and supports approximate search. The system automatically updates the root vector when the data changes, ensuring that the tree structure remains balanced and optimized for modern vector (embedding) processing applications.

4 Requirements and Grading

4.1 Requirements

To complete this assignment, students need to:

1. Read this entire description file carefully.
2. Download the **initial.zip** file and extract it. After extracting, students will obtain the following files: `utils.h`, `main.cpp`, `main.h`, `VectorStore.h`, `VectorStore.cpp`, and a folder containing sample outputs. Students only need to submit two files: `VectorStore.h` and `VectorStore.cpp`. Therefore, students are not allowed to modify the `main.h` file when testing the program.
3. Use the following command to compile:
   `g++ -o main main.cpp VectorStore.cpp -I . -std=c++17`
   This command should be used in Command Prompt/Terminal to compile the program. If students use an IDE to run the program, note that they must: add all files to the IDE's project/workspace; modify the build command in the IDE accordingly. IDEs usually provide a Build button and a Run button. When clicking Build, the IDE runs the corresponding compile command, which typically compiles only `main.cpp`. Students must configure the compile command to include `VectorStore.cpp`, and add the options `-std=c++17` and `-I .`
4. The program will be graded on a Unix-based platform. Students' environments and compilers may differ from the actual grading environment. The submission area on LMS is configured similarly to the grading environment. Students must check their program on the submission page and fix all errors reported by LMS to ensure correct final results.

5. Edit the `VectorStore.h` and `VectorStore.cpp` files to complete the assignment, while ensuring the following two requirements:
   - All methods described in this guide must be implemented so that the program can compile successfully. If a method has not yet been implemented, students must provide an empty implementation for that method. Each test case will call certain methods to check their return values.
   - The file `VectorStore.h` must contain exactly one line `#include "main.h"`, and the file `VectorStore.cpp` must contain exactly one line `#include "VectorStore.h"`. Apart from these, no other `#include` statements are allowed in these files.
   - Students are not allowed to use the directive `#define TESTING` in the two files that are required to be modified.
6. Students are encouraged to write additional supporting classes, methods, and attributes within the classes they are required to implement. However, these additions must not change the requirements of the methods described in the assignment.
7. Students must design and use data structures learned in the course.
8. Students must ensure that all dynamically allocated memory is properly freed when the program terminates.

4.2 Submission Deadline

The deadline is **as announced on LMS**. Students must submit their assignments to the system before the specified deadline. Students are fully responsible for any issues arising from submissions made too close to the deadline.

4.3 Grading

All student source code will be evaluated using hidden test cases, and scores will be calculated based on each specific requirement.

5 Harmony Questions

The final exam for the course will include several "Harmony" questions related to the content of the Assignment.

Students must complete the Assignment by their own ability. If a student cheats in the

Assignment, they will not be able to answer the Harmony questions and will receive a score of 0 for the Assignment.

Students **must** pay attention to completing the Harmony questions in the final exam. Failing to do so will result in a score of 0 for the Assignment, and the student will fail the course. **No explanations and no exceptions.**

6 Regulations and Handling of Cheating

The Assignment must be done by the student THEMSELVES. A student will be considered cheating if:

- There is an unusual similarity between the source code of submitted projects. In this case, ALL submissions will be considered as cheating. Therefore, students must protect their project source code.
- The student does not understand the source code they have written, except for the parts of code provided in the initialization program. Students can refer to any source of material, but they must ensure they understand the meaning of every line of code they write. If they do not understand the source code from where they referred, the student will be specifically warned NOT to use this code; instead, they should use what has been taught to write the program.
- Submitting someone else's work under their own account.
- Students use AI tools during the Assignment process, resulting in identical source code.

If the student is concluded to be cheating, they will receive a score of 0 for the entire course (not just the assignment).

NO EXPLANATIONS WILL BE ACCEPTED AND THERE WILL BE NO EXCEPTIONS!

After the final submission, some students will be randomly selected for an interview to prove that the submitted project was done by them.

Other regulations:

- All decisions made by the lecturer in charge of the assignment are final decisions.
- Students are not provided with test cases after the grading of their project.

- The content of the Assignment will be harmonized with questions in the exam that has similar content.

——————————THE END——————————